SPD1179 CAN 使用指南

概述

控制器局域网络（CAN）是一种应用广泛的串行通信协议，常用于汽车电子、工业自动化领域。SPD1179 内置一个 CAN 模块，提供多种功能以及波特率和采样点的配置，兼容 ISO 11898-1:2015、BOSCH CAN2.0B、BOSCH FDCAN V1.0 标准，为 UDS、OBD 等应用层协议提供物理层和数据链路层的支持。

目录

图片列表

表格列表

未找到图形项目表。

版本历史

版本	日期	作者	状态	变更
C/0	2024-02-06	高天骥	Released	首次发布。

术语或缩写

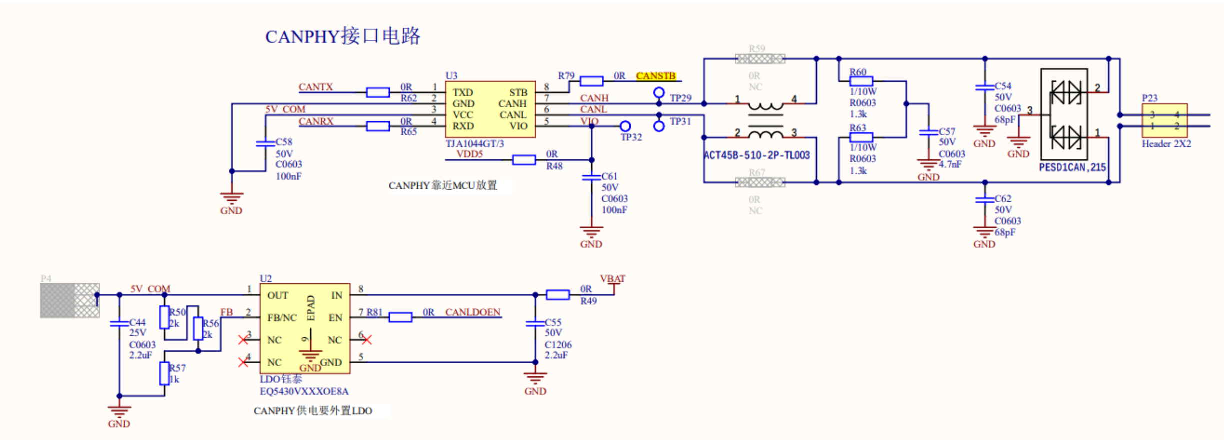
术语或缩写	描述
MCU	Microcontroller Unit，微控制器单元
CAN	Controller Area Network，控制器局域网总线
CAN FD	CAN with Flexible Data rate，可变速率的控制器局域网总线

1 SPD1179 Demo 板硬件配置

1.1 CAN PHY 接口电路

48-Pin SPD1179 Demo 板的 CAN PHY 接口电路的原理图如图 1-1 中所示。Demo 板使用的 CAN PHY 型号为 TJA1044GT/3，支持 Standby 模式和 CANFD。CAN PHY 使用一个外置的 5V LDO 供电。

图 1-1：Demo 板 CAN PHY 接口电路



根据原理图可知，要使 CAN PHY 接口电路正常工作，需要做如下配置：

- 使用外接电源给 Demo 板正常供电；
- CAN 模块的 Tx、Rx 引脚分别配置为 GPIO_8、GPIO_9；
- CANLDOEN(GPIO7)输出高电平，使能 LDO；
- CANSTB(GPIO6)输出低电平，关闭 CAN PHY 的 Standby 模式。

2 SPD1179 CAN 模块软件配置

2.1 CAN 模块例程

如图 2-1 所示，SPD1179 的 SDK 中包含了 6 个 CAN 模块相关的例程，展示了 CANFD、CAN 扩展帧等功能的配置方法。用户可以根据自己的需求选择合适的例程。

图 2-1：SDK 中 CAN 模块相关例程

26_1_CAN_Extended_Frame_RX
26_1_CAN_Standard_And_Extended_Frame_TX
26_1_CAN_Standard_Frame_RX
26_2_CANFD_Extended_Frame_RX
26_2_CANFD_Standard_And_Extended_Frame_TX
26_2_CANFD_Standard_Frame_RX

2.2 GPIO 配置

SDK 中 CAN 模块相关例程中默认的 CAN_Tx、CAN_Rx 引脚配置如图 2-2 中所示，与 Demo 板上硬件的引脚接线不一致。因此，需要使 GPIO7 输出高电平，从而使能 LDO，并使 GPIO6 输出低电平，从而关闭 CAN PHY 的 Standby 模式。另外，还需要将 CAN_Tx、CAN_Rx 分别配置为 GPIO_8、GPIO_9。修改后的引脚配置如图 2-3 中所示。

图 2-2：例程中默认的引脚配置

main.c

```c
static void CAN_Gpio_Init(void)
{
    #if defined(SPD1179)
    PIN_SetChannel(PIN_GPIO16, PIN_GPIO16_CAN_TXD);
    PIN_SetChannel(PIN_GPIO17, PIN_GPIO17_CAN_RXD);
    #else
    PIN_SetChannel(PIN_GPIO38, PIN_GPIO38_CAN_TXD);
    PIN_SetChannel(PIN_GPIO39, PIN_GPIO39_CAN_RXD);
    #endif
}
```

图 2-3：修改后的引脚配置

main.c

```c
static void CAN_Gpio_Init(void)
{
    #if defined(SPD1179)
    PIN_SetChannel(PIN_GPIO8, PIN_GPIO8_CAN_TXD);
    PIN_SetChannel(PIN_GPIO9, PIN_GPIO9_CAN_RXD);
    PIN_SetPinAsGPIO(PIN_GPIO6);
    GPIO_SetPinDir(PIN_GPIO6, GPIO_OUTPUT);
    GPIO_SetLow(PIN_GPIO6); // Set CAN PHY to normal mode (STB Pin level low)
    PIN_SetPinAsGPIO(PIN_GPIO7);
    GPIO_SetPinDir(PIN_GPIO7, GPIO_OUTPUT);
    GPIO_SetHigh(PIN_GPIO7);     // Enable LDO
    #else
    PIN_SetChannel(PIN_GPIO38, PIN_GPIO38_CAN_TXD);
    PIN_SetChannel(PIN_GPIO39, PIN_GPIO39_CAN_RXD);
    #endif
}
```

2.3 CAN 模块配置

CAN 模块的多项功能需要在模块使能之前进行配置，包括 TXD/RXD 引脚交换、CANFD、BOSCH CAN/ISO CAN、协议异常事件检测、限制模式、监控模式、发送暂停、边沿滤波、自动重发、总线自动恢复等功能。每个功能的具体描述请参考《SPD1179 技术参考手册》。下面的代码展示了例程中使用的 CAN 模块配置，用户可以根据实际的需求修改配置。

Example Code

```
    /* Set IO swap */
    CAN_DisablePinSwap(CAN);              //关闭 TXD/RXD 引脚交换

    /* Set FDCAN fromat support */
    CAN_DisableFDFormat(CAN);             //关闭 CANFD

    /* Set ISO-CAN frame support */
    CAN_DisableNonIsoMode(CAN);           //ISO CAN

    /* Set protocol exception mode */
    CAN_EnableProtocolException(CAN);    //开启协议异常事件检测，FDF 位为隐性时触发异常事
件

    /* Set restricted mode */
    CAN_DisableRestrictedMode(CAN);      //关闭限制模式

    /* Set monitor mode */
    CAN_DisableMonitorMode(CAN);          //关闭监控模式

    /* Set transmission pause function */
    CAN_EnableTransmissionPause(CAN);  //开启发送暂停，在两个连续帧之间插入两个 bit 时间

    /* Set RXD edge filter during integration status */
    CAN_EnableEdgeFilter(CAN);            //开启边沿滤波，隐性位后出现两个连续的显性位才判
定为下降沿

    /* Enable re-tran message afte lost arbitration or error */
    CAN_EnableAutoRetransmission(CAN);  //开启自动重发

    /* Enable auto start bus-off recovery sequence after bus-off */
    CAN_EnableAutoBusOn(CAN);             //开启总线自动恢复
```

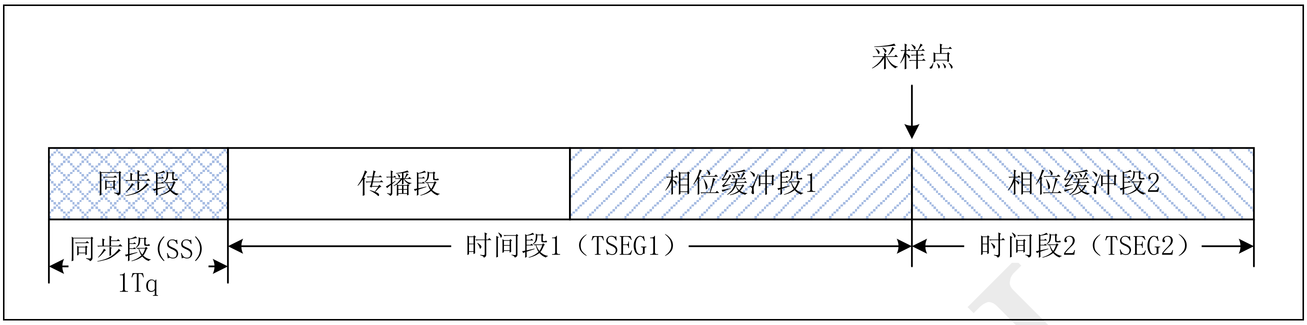
2.4 比特时间和时间量子

在 CAN 通信中，每一个数据位持续的时间称为比特时间，比特时间的基本单位称为时间量子，即每个比特时间由 N 个时间量子（Tq）组成，同时每个比特时间又可以划分为 4 段，分别为：

1. 同步段（SS: Synchronization Segment）；
2. 传播时间段（PTS: Propagation Time Segment）；
3. 相位缓冲段 1（PBS1: Phase Buffer Segment 1）；
4. 相位缓冲段 2（PBS2: Phase Buffer Segment 2）。

SPD1179 的 CAN 模块将传播段与相位缓冲段 1 合并为一个时间段，称为时间段 1（TSEG1），相位缓冲段 2 称为（TSEG2），因此比特时间的构成如图 2-4 所示。

图 2-4：比特时间构成

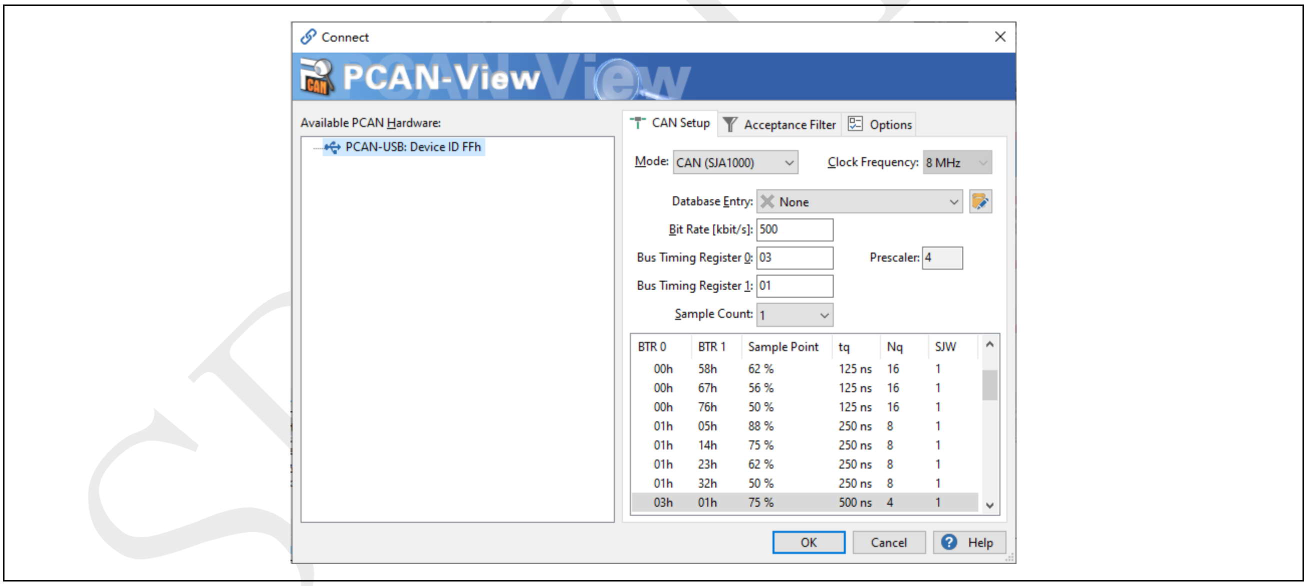


2.5 采样点和波特率设置

SPD1179 可以通过修改 CAN 模块的时钟频率、时间段 1 的长度、时间段 2 的长度三个参数，达到设置 CAN 通信的采样点和波特率的目的。

下面将通过一个实例说明时钟频率、时间段 1 的长度、时间段 2 如何设置。首先，需要知道 CAN 上位机的参数，此处上位机的波特率为 500k，采样点为 75%，如图 2-5 所示。

图 2-5：上位机的波特率和采样点



为保证 CAN 总线的正常通信，SPD1179 的 CAN 模块的参数需要尽量与上位机保持一致，即波特率 500k，采样点 75%。因此，需要在程序中做如下设置，如图 2-6 中所示。例程中 CPU 频率为 100MHz，此处将 CAN 模块的时钟频率设置为 CPU 时钟频率的 50 分频，即 2MHz。时间量子 Tq 的长度就是 CAN 模块一个时钟周期的长度，即 500ns。同步段的长度默认为 1 个 Tq，时间段 1 的长度设置为 2 个 Tq，时间段 2 的长度设置为 1 个 Tq。此时，1 个比特的时间长度就是（1+2+1）×500ns = 2us，即波特率为 500kHz。由于采样点位于时间段 1 与时间段 2 交界处，因此采样点为（1+2）/（1+2+1）= 75%。

实际应用中的需求与本实例中不同时，需要用户参照上述步骤，自行计算出合适的 CAN 模块的时钟频率、时间段 1 的长度、时间段 2 的长度三个参数，从而保证 SPD1179 的 CAN 模块与总线上其余设备能够正常通信。

图 2-6：标称位采样点和波特率设置

main.c | spd1179_hv_bitfield.h | spd1179_hv.c | epwr.c | can.h

```c
/* Set nominal bit baud rate
 * 500kHz for 100MHz clock
 **/
CAN_SetNominalBitRatePrescaler(CAN, 50);          //Tq = 1/(100M/50) = 500ns, SS = 1 * 500ns = 500ns
CAN_SetNominalBitPhaseBufferSegment1(CAN, 2);     //TSEG1 = 2 * 500ns = 1us
CAN_SetNominalBitPhaseBufferSegment2(CAN, 1);     //TSEG2 = 1 * 500ns = 500ns
CAN_SetNominalBitResyncJumpWidth(CAN, 1);
```

如果开启了 CANFD，还需要配置数据位的波特率，如图 2-7 所示。

图 2-7：数据位采样点和波特率设置

```c
/* Set data bit baud rate */
if ( CAN_IsEnableFDFormat(CAN))
{
    u32Prescaler = CAN_GetNominalBitRatePrescaler(CAN);
    /* 5Mbps for 100MHz clock */
    CAN_SetDataBitRatePrescaler(CAN, u32Prescaler);
    CAN_SetDataBitPhaseBufferSegment1(CAN, 17 );
    CAN_SetDataBitPhaseBufferSegment2(CAN, 2 ); /* sample point: 80% */
    CAN_SetDataBitResyncJumpWidth    (CAN, 2 );
}
```

2.6 CAN 邮箱配置

SPD1179 的 CAN 模块有 64 个邮箱，每个邮箱都可以单独配置邮箱的 CAN ID、邮箱方向（接收或发送报文）、协议类型（CAN 或 CANFD）、帧格式（标准帧或扩展帧）、帧类型（远程帧或数据帧）等工作模式，邮箱的配置主要分为以下几个步骤：

1. 创建对应邮箱的结构体变量；
2. 按照需求配置邮箱的工作模式和功能；
3. 使能邮箱。

下面将对邮箱的常用工作模式做简单介绍，详细信息可以参考《SPD1179 技术参考手册》和《ISO-11898-1:2015》标准：

- CAN ID：CAN ID 用于优先级的仲裁，更小的 CAN ID 拥有更高的优先级。为避免总线冲突，通常每个节点只能通过独有的 CAN ID 发送帧，但可以接收多个不同 CAN ID 的帧。
- 邮箱方向：每个邮箱都只能选择一个方向，发送报文或接收报文。
- 协议类型：CAN 协议帧中的位可以分为标称位（Norminal bit）和数据位（Data bit）。经典 CAN 协议中标称位和数据位的波特率始终保持一致，而 CANFD 协议中数据位可以使用更高的波特率。使用 CANFD 功能时，在正确配置邮箱之外，CAN 模块也要进行相应配置。
- 帧格式：CAN 协议支持标准帧和扩展帧两种帧格式，主要区别在于标准帧的 CAN ID 为 11 位，而扩展帧的 CAN ID 为 29 位。

– 帧类型：CAN 协议支持远程帧和数据帧两种帧类型，主要区别在于远程帧不包含数据位，而数据帧包含了最多 8 个字节的数据。远程帧一般用于请求数据，节点需要在收到远程帧后返回数据帧。这种方式的好处在于，远程帧和数据帧可以使用同一个 CAN ID 的同时，又因为数据帧具有更高的优先级，从而避免了总线冲突。

本章后续部分将展示不同工作模式下邮箱的配置方法，用户还可以参考对应例程和《SPD1179 技术参考手册》中“21.6 信箱控制”章节的内容。

2.6.1 发送时的邮箱配置

2.6.1.1 标准远程帧

下面的代码展示了发送 CAN 标准远程帧时对邮箱的配置。

Example Code

```
……

    /* CAN Standard Remote frame message1 */
    CAN_MessageTypeDef message1;    //创建对应邮箱的结构体变量

……

    /* Init mailbox for transmit CAN Standard Data frame message1 */
    message1.eDataDir    = CAN_MSG_DATA_TX; //邮箱方向为发送
    message1.eRmtRspEn   = DISABLE;         //关闭自动回复远程帧
    message1.eIntEn      = ENABLE;          //开启发送完成中断
    message1.eEobEn      = DISABLE;         //当前邮箱不是邮箱块中最后一个
    message1.eBrs        = DISABLE;         //关闭速率切换
    message1.eEsiCtlEn   = ENABLE;          //ESI 由 CANMBOXMCTL.ESI 寄存器位段控制
    message1.eEsi        = DISABLE;         //被动错误

    message1.u32Id       = 0x11;            //帧 ID
    message1.u8MBoxId    = 0;               //邮箱号
    message1.eFormat     = CAN_FORMAT_STD;  //标准帧
    message1.eType       = CAN_FRAME_REMOTE;//远程帧

    /* Remote frame no DATA FIELD */
    message1.u8DataLen = 0;                 //数据长度
    message1.pu8Data     = NULL;            //数据 Buffer 的指针

    status = CAN_SetMessage(CAN, &message1);//根据 message1 结构体配置邮箱
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message1.u8MBoxId);
        return -1;
    }

    CAN_EnableMailbox(CAN, message1.u8MBoxId);//使能邮箱，发送 CAN 协议帧

    /* Wait CAN Standard Remote frame message1 txed */
    while (CAN_GetMailboxControlInfo(CAN, message1.u8MBoxId,
CAN_MBOX_SET_MSG_NEW | CAN_MBOX_REQUEST_TX)) {}//等待发送完成

    printf("\nTransmit CAN Standard Remote frame success\n");
```

2.6.1.2 标准数据帧

下面的代码展示了发送 CAN 标准数据帧时对邮箱的配置。若开启了 CANFD，则需要将变量“eFormat”的值配置为宏“CAN_FORMAT_FD_STD”。

Example Code

```
……

    /* CAN Standard Data frame message2 */
    CAN_MessageTypeDef message2;    //创建对应邮箱的结构体变量

……

    /* Init mailbox for transmit CAN Standard Data frame message2 */
    message2.eDataDir    = CAN_MSG_DATA_TX; //邮箱方向为发送
    message2.eRmtRspEn   = DISABLE;         //关闭自动回复远程帧
    message2.eIntEn      = ENABLE;          //开启发送完成中断
    message2.eEobEn      = DISABLE;         //当前邮箱不是邮箱块中最后一个
    message2.eBrs        = DISABLE;         //关闭速率切换
    message2.eEsiCtlEn   = ENABLE;          //ESI 由 CANMBOXMCTL.ESI 寄存器位段控制
    message2.eEsi        = DISABLE;         //被动错误

    message2.u32Id       = 0x22;            //帧 ID
    message2.u8MBoxId    = 1;               //邮箱号
    message2.eFormat     = CAN_FORMAT_STD;  //标准帧，CANFD 需改为 CAN_FORMAT_FD_STD
    message2.eType       = CAN_FRAME_DATA;  //数据帧

    /* DATA FIELD */
    message2.u8DataLen   = CAN_DATA_LEN;    //数据长度
    message2.pu8Data     = au8TxBuffer;     //数据 Buffer 的指针

    for (i = 0; i < message2.u8DataLen; i++)
    {
        message2.pu8Data[i] = i;            //将数据写入 Buffer
    }

    #if DEBUG_INFO
    printf("CAN Standard Frame Data:\n") ;
    CAN_Printf_Message(&message2);
    #endif

    status = CAN_SetMessage(CAN, &message2);//根据 message2 结构体配置邮箱
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message2.u8MBoxId);
        return -1;
    }

    CAN_EnableMailbox(CAN, message2.u8MBoxId);//使能邮箱，发送 CAN 协议帧
    /* Wait CAN Standard Data frame message2 txed */

    while (CAN_GetMailboxControlInfo(CAN, message2.u8MBoxId,
CAN_MBOX_SET_MSG_NEW | CAN_MBOX_REQUEST_TX)) {}//等待发送完成

    printf("\nTransmit CAN Standard Data frame success\n");
```

2.6.1.3 扩展远程帧

下面的代码展示了发送 CAN 扩展远程帧时对邮箱的配置。

Example Code

```
……

    /* CAN Standard Data frame message3 */
    CAN_MessageTypeDef message3;    //创建对应邮箱的结构体变量

……

    /* Init mailbox for transmit CAN Extended Remote frame message3 */
    message3.eDataDir    = CAN_MSG_DATA_TX; //邮箱方向为发送
    message3.eRmtRspEn   = DISABLE;         //关闭自动回复远程帧
    message3.eIntEn      = ENABLE;          //开启发送完成中断
    message3.eEobEn      = DISABLE;         //当前邮箱不是邮箱块中最后一个
    message3.eBrs        = DISABLE;         //关闭速率切换
    message3.eEsiCtlEn   = ENABLE;          //ESI 由 CANMBOXMCTL.ESI 寄存器位段控制
    message3.eEsi        = DISABLE;         //被动错误

    message3.u32Id       = 0x33;            //帧 ID
    message3.u8MBoxId    = 3;               //邮箱号
    message3.eFormat     = CAN_FORMAT_EXT;  //扩展帧
    message3.eType       = CAN_FRAME_REMOTE;//远程帧

    /* Remote frame no DATA FIELD */
    message3.u8DataLen   = 0;               //数据长度
    message3.pu8Data     = NULL;            //数据 Buffer 的指针

    status = CAN_SetMessage(CAN, &message3);//根据 message3 结构体配置邮箱
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message3.u8MBoxId);
        return -1;
    }

    CAN_EnableMailbox(CAN, message3.u8MBoxId);//使能邮箱，发送 CAN 协议帧

    /* Wait CAN Extended Remote frame message3 txed */
    while (CAN_GetMailboxControlInfo(CAN, message3.u8MBoxId,
CAN_MBOX_SET_MSG_NEW | CAN_MBOX_REQUEST_TX)) {}//等待发送完成

    printf("\nTransmit CAN Extended Remote frame success\n");
```

2.6.1.4 扩展数据帧

下面的代码展示了发送 CAN 扩展数据帧时对邮箱的配置。若开启了 CANFD，则需要将变量“eFormat”的值配置为宏“CAN_FORMAT_FD_EXT”。

Example Code

```
......

    /* CAN Standard Data frame message4 */
    CAN_MessageTypeDef message4;    //创建对应邮箱的结构体变量

......

    /* Init mailbox for transmit CAN Extended Data frame message4 */
    message4.eDataDir    = CAN_MSG_DATA_TX; //邮箱方向为发送
    message4.eRmtRspEn   = DISABLE;         //关闭自动回复远程帧
    message4.eIntEn      = ENABLE;          //开启发送完成中断
    message4.eEobEn      = DISABLE;         //当前邮箱不是邮箱块中最后一个
    message4.eBrs        = DISABLE;         //关闭速率切换
    message4.eEsiCtlEn   = ENABLE;          //ESI 由 CANMBOXMCTL.ESI 寄存器位段控制
    message4.eEsi        = DISABLE;         //被动错误

    message4.u32Id       = 0x44;            //帧 ID
    message4.u8MBoxId    = 4;               //邮箱号
    message4.eFormat     = CAN_FORMAT_EXT;  //扩展帧，CANFD 需改为 CAN_FORMAT_FD_EXT
    message4.eType       = CAN_FRAME_DATA;  //数据帧

    /* DATA FIELD */
    message4.u8DataLen   = CAN_DATA_LEN;    //数据长度
    message4.pu8Data     = au8TxBuffer;     //数据 Buffer 的指针

    for (i = 0; i < message4.u8DataLen; i++)
    {
        message4.pu8Data[i] = i;            //将数据写入 Buffer
    }

    #if DEBUG_INFO
    printf("CAN Extended Frame Data:\n") ;
    CAN_Printf_Message(&message4);
    #endif

    status = CAN_SetMessage(CAN, &message4);//根据 message4 结构体配置邮箱
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message4.u8MBoxId);
        return -1;
    }

    CAN_EnableMailbox(CAN, message4.u8MBoxId);//使能邮箱，发送 CAN 协议帧

    /* Wait CAN Extended Data frame message4 txed */
    while (CAN_GetMailboxControlInfo(CAN, message4.u8MBoxId,
CAN_MBOX_SET_MSG_NEW | CAN_MBOX_REQUEST_TX)) {}//等待发送完成

    printf("\nTransmit CAN Extended Data frame success\n");
```

2.6.2 接收时的邮箱配置

2.6.2.1 标准远程帧

下面的代码展示了接收 CAN 标准远程帧时对邮箱的配置。

Example Code

```
……

    /* CAN Standard Data frame message1 */
    CAN_MessageTypeDef message1;    //创建对应邮箱的结构体变量

……

    /* Init mailbox for receive message1 */
    message1.eDataDir     = CAN_MSG_DATA_RX; //邮箱方向为接收
    message1.eRmtRspEn     = DISABLE;         //关闭自动回复远程帧
    message1.eIntEn       = ENABLE;          //打开接收完成中断
    message1.eEobEn       = DISABLE;         //当前邮箱不是邮箱块中最后一个
    message1.eOverwriteEn= ENABLE;          //邮箱块最后一个邮箱改写使能，EobEn 为 1 时生效
    message1.eMaskRtrEn   = ENABLE;          //ID 过滤匹配时 RTR 位参与比较
    message1.eMaskIdeEn   = ENABLE;          //ID 过滤匹配时 IDE 位参与比较

    message1.u32Id        = 0x11;            //帧 ID
    message1.u32IdMask     = 0xff;            //帧 ID 掩码
    message1.u8MBoxId     = 0;               //邮箱号
    message1.eFormat      = CAN_FORMAT_STD;  //标准帧
    message1.eType        = CAN_FRAME_REMOTE;//远程帧

    /* Remote frame no DATA FIELD */
    message1.pu8Data      = NULL;            //数据 Buffer 的指针
    message1.u8DataLen     = 0;               //数据长度

    status = CAN_SetMessage(CAN, &message1);//根据 message1 结构体配置邮箱
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message1.u8MBoxId);
        return -1;
    }

    CAN_EnableMailbox(CAN, message1.u8MBoxId);//使能邮箱

    /* Wait receive CAN Standard Remote frame message1 */
    while (!CAN_GetMailboxControlInfo(CAN, message1.u8MBoxId,
CAN_MBOX_SET_MSG_NEW)) {}//等待接收

    printf("FLTOK:%d\n", CAN_GetMiscIntRawFlag(CAN, 4)); //read CANMRAWIF.FLTOK
    printf("CANMSGNEW:%d\n", CAN_GetMailbox0To31NewMessageFlag(CAN, 1)); //read
mbx0 new msg flg

    CAN_GetMessage( CAN, &message1);        //获取帧

    printf("\nReceives CAN Standard Remote frame success\n");
```


2.6.2.2 标准数据帧

下面的代码展示了接收 CAN 标准数据帧时对邮箱的配置。若开启了 CANFD，则需要将变量“eFormat”的值配置为宏“CAN_FORMAT_FD_STD”。

Example Code

```
……

    /* CAN Standard Data frame message2 */
    CAN_MessageTypeDef message2;    //创建对应邮箱的结构体变量

……

    /* Init mailbox for receive message1 */
    message2.eDataDir    = CAN_MSG_DATA_RX; //邮箱方向为接收
    message2.eRmtRspEn   = DISABLE;         //关闭自动回复远程帧
    message2.eIntEn      = ENABLE;          //打开接收完成中断
    message2.eEobEn      = DISABLE;         //当前邮箱不是邮箱块中最后一个
    message2.eOverwriteEn= ENABLE;         //邮箱块最后一个邮箱改写使能，EobEn 为 1 时生效
    message2.eMaskRtrEn  = ENABLE;          //ID 过滤匹配时 RTR 位参与比较
    message2.eMaskIdeEn  = ENABLE;          //ID 过滤匹配时 IDE 位参与比较

    message2.u32Id       = 0x22;            //帧 ID
    message2.u32IdMask   = 0xff;            //帧 ID 掩码
    message2.u8MBoxId    = 1;              //邮箱号
    message2.eFormat     = CAN_FORMAT_STD;  //扩展帧，CANFD 需改为 CAN_FORMAT_FD_STD
    message2.eType       = CAN_FRAME_DATA;  //远程帧

    /* DATA FIELD */
    message2.u8DataLen   = CAN_DATA_LEN;              //数据长度

    status = CAN_SetMessage(CAN, &message2);//根据 message2 结构体配置邮箱
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message2.u8MBoxId);
        return -1;
    }

    message2.pu8Data = (uint8_t *)(long)(& CAN->
CANMBOX[message2.u8MBoxId].CANMBOXFDW[0]);  //数据指针指向邮箱报文数据字寄存器首地址

    CAN_EnableMailbox(CAN, message2.u8MBoxId);//使能邮箱

    /* Wait receive CAN Standard Remote frame message1 */
    while (!CAN_GetMailboxControlInfo(CAN, message2.u8MBoxId,
CAN_MBOX_SET_MSG_NEW)) {}//等待接收

    printf("FLTOK:%d\n", CAN_GetMiscIntRawFlag(CAN, 4)); //read CANMRAWIF.FLTOK
    printf("CANMSGNEW:%d\n", CAN_GetMailbox0To31NewMessageFlag(CAN, 1)); //read
mbx0 new msg flg

    CAN_GetMessage( CAN, &message2);       //获取帧
```

2.6.2.3 扩展远程帧

下面的代码展示了接收 CAN 扩展远程帧时对邮箱的配置。

Example Code

```
……

    /* CAN Standard Data frame message1 */
    CAN_MessageTypeDef message1;    //创建对应邮箱的结构体变量

……

    /* Init mailbox for receive message1 */
    message1.eDataDir     = CAN_MSG_DATA_RX; //邮箱方向为接收
    message1.eRmtRspEn    = DISABLE;         //关闭自动回复远程帧
    message1.eIntEn       = ENABLE;          //打开接收完成中断
    message1.eEobEn       = DISABLE;         //当前邮箱不是邮箱块中最后一个
    message1.eOverwriteEn= ENABLE;          //邮箱块最后一个邮箱改写使能，EobEn 为 1 时生效
    message1.eMaskRtrEn   = ENABLE;          //ID 过滤匹配时 RTR 位参与比较
    message1.eMaskIdeEn   = ENABLE;          //ID 过滤匹配时 IDE 位参与比较

    message1.u32Id        = 0x33;            //帧 ID
    message1.u32IdMask    = 0xff;            //帧 ID 掩码
    message1.u8MBoxId     = 0;               //邮箱号
    message1.eFormat      = CAN_FORMAT_EXT;  //标准帧
    message1.eType        = CAN_FRAME_REMOTE;//远程帧

    /* Remote frame no DATA FIELD */
    message1.pu8Data      = NULL;            //数据 Buffer 的指针
    message1.u8DataLen    = 0;               //数据长度

    status = CAN_SetMessage(CAN, &message1);//根据 message1 结构体配置邮箱
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message1.u8MBoxId);
        return -1;
    }

    CAN_EnableMailbox(CAN, message1.u8MBoxId);//使能邮箱

    /* Wait receive CAN Standard Remote frame message1 */
    while (!CAN_GetMailboxControlInfo(CAN, message1.u8MBoxId,
CAN_MBOX_SET_MSG_NEW)) {}//等待接收

    printf("FLTOK:%d\n", CAN_GetMiscIntRawFlag(CAN, 4)); //read CANMRAWIF.FLTOK
    printf("CANMSGNEW:%d\n", CAN_GetMailbox0To31NewMessageFlag(CAN, 1)); //read
mbx0 new msg flg

    CAN_GetMessage( CAN, &message1);        //获取帧

    printf("\nReceives CAN Standard Remote frame success\n");
```


2.6.2.4 扩展数据帧

下面的代码展示了接收 CAN 标准数据帧时对邮箱的配置。若开启了 CANFD，则需要将变量“eFormat”的值配置为宏“CAN_FORMAT_FD_EXT”。

Example Code

```
……

    /* CAN Standard Data frame message2 */
    CAN_MessageTypeDef message2;    //创建对应邮箱的结构体变量

……

    /* Init mailbox for receive message1 */
    message2.eDataDir    = CAN_MSG_DATA_RX; //邮箱方向为接收
    message2.eRmtRspEn   = DISABLE;        //关闭自动回复远程帧
    message2.eIntEn      = ENABLE;         //打开接收完成中断
    message2.eEobEn      = DISABLE;        //当前邮箱不是邮箱块中最后一个
    message2.eOverwriteEn= ENABLE;        //邮箱块最后一个邮箱改写使能，EobEn 为 1 时生效
    message2.eMaskRtrEn  = ENABLE;         //ID 过滤匹配时 RTR 位参与比较
    message2.eMaskIdeEn  = ENABLE;         //ID 过滤匹配时 IDE 位参与比较

    message2.u32Id       = 0x44;          //帧 ID
    message2.u32IdMask   = 0xff;          //帧 ID 掩码
    message2.u8MBoxId    = 1;             //邮箱号
    message2.eFormat     = CAN_FORMAT_EXT;  //扩展帧，CANFD 需改为 CAN_FORMAT_FD_EXT
    message2.eType       = CAN_FRAME_DATA;  //远程帧

    /* DATA FIELD */
    message2.u8DataLen   = CAN_DATA_LEN;          //数据长度

    status = CAN_SetMessage(CAN, &message2);//根据 message2 结构体配置邮箱
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message2.u8MBoxId);
        return -1;
    }

    message2.pu8Data = (uint8_t *)(long)(& CAN->
CANMBOX[message2.u8MBoxId].CANMBOXFDW[0]);  //数据指针指向邮箱报文数据字寄存器首地址

    CAN_EnableMailbox(CAN, message2.u8MBoxId);//使能邮箱

    /* Wait receive CAN Standard Remote frame message1 */
    while (!CAN_GetMailboxControlInfo(CAN, message2.u8MBoxId,
CAN_MBOX_SET_MSG_NEW)) {}//等待接收

    printf("FLTOK:%d\n", CAN_GetMiscIntRawFlag(CAN, 4)); //read CANMRAWIF.FLTOK
    printf("CANMSGNEW:%d\n", CAN_GetMailbox0To31NewMessageFlag(CAN, 1)); //read
mbx0 new msg flg

    CAN_GetMessage( CAN, &message2);       //获取帧
```

3 CAN 通信实例

3.1 经典 CAN 协议

下面将展示采用经典 CAN 协议的通信实例，总线上包含两个节点：上位机（连接 PC 的 USB CAN）和 SPD1179。上位机依次发送一条标准数据帧和一条扩展数据帧，SPD1179 将收到的每个 Byte 数据“+1”后，以同样的格式发回上位机。实例代码如下所示，通信结果如图 3-1 所示。

Example Code

```
#include <stdio.h>
#include <stdlib.h>

#if defined(SPD1179)
    #include "spd1179.h"
#else
    #include "spc1169.h"
#endif


#define DEBUG_INFO      1

#define CAN_DATA_LEN       8

uint8_t au8TxBuffer[64];

/* CAN Standard Data frame message1 */
CAN_MessageTypeDef message1;
/* CAN Extended Data frame message2 */
CAN_MessageTypeDef message2;
/* CAN reply data frame message3 */
CAN_MessageTypeDef message3;

static void CAN_Clk_Enable(void)
{
    /* CAN Clk Enable */
    CLOCK_SetModuleDiv(CAN_MODULE, 1);
    CLOCK_EnableModule(CAN_MODULE);
}


static void CAN_Module_Disable(void)
{
    /* Disable run */
    CAN_Disable(CAN) ;

    /* FIXME add timeout control */
    while (CAN_GetOperationMode(CAN) != CAN_OPERATE_STOP) {}
}

static void CAN_Gpio_Init(void)
{
    #if defined(SPD1179)
    PIN_SetChannel(PIN_GPIO8, PIN_GPIO8_CAN_TXD);
    PIN_SetChannel(PIN_GPIO9, PIN_GPIO9_CAN_RXD);
    PIN_SetPinAsGPIO(PIN_GPIO6);
    GPIO_SetPinDir(PIN_GPIO6, GPIO_OUTPUT);
    GPIO_SetLow(PIN_GPIO6); // Set CAN PHY to normal mode (STB Pin level low)
```

```
    PIN_SetPinAsGPIO(PIN_GPIO7);
    GPIO_SetPinDir(PIN_GPIO7, GPIO_OUTPUT);
    GPIO_SetHigh(PIN_GPIO7);    // Enable LDO
    #else
    PIN_SetChannel(PIN_GPIO38, PIN_GPIO38_CAN_TXD);
    PIN_SetChannel(PIN_GPIO39, PIN_GPIO39_CAN_RXD);
    #endif
}

static void CAN_Config(void)
{
    uint32_t u32Data ;
    uint32_t u32DateSegment1 ;

    double f64NBps ;
    double f64DBps ;

    uint32_t u32Prescaler;

    /* Set IO swap */
    CAN_DisablePinSwap(CAN) ;

    /* Set FDCAN fromat support */
    CAN_DisableFDFormat(CAN);

    /* Set ISO-CAN frame support */
    CAN_DisableNonIsoMode(CAN);

    /* Set protocol exception mode */
    CAN_EnableProtocolException(CAN);

    /* Set restricted mode */
    CAN_DisableRestrictedMode(CAN);

    /* Set monitor mode */
    CAN_DisableMonitorMode(CAN);

    /* Set transmission pause function */
    CAN_EnableTransmissionPause(CAN);

    /* Set RXD edge filter during integration status */
    CAN_EnableEdgeFilter(CAN);

    /* Enable re-tran message afte lost arbitration or error */
    CAN_EnableAutoRetransmission(CAN);

    /* Enable auto start bus-off recovery sequence after bus-off */
    CAN_EnableAutoBusOn(CAN);

    /* Set nominal bit baud rate
     * 1MHz for 100MHz clock
     **/
    CAN_SetNominalBitRatePrescaler(CAN, 50);
    CAN_SetNominalBitPhaseBufferSegment1(CAN, 2);
    CAN_SetNominalBitPhaseBufferSegment2(CAN, 1); /* sample point: 83% */
    CAN_SetNominalBitResyncJumpWidth(CAN, 1);

    /* Set data bit baud rate */
    if ( CAN_IsEnableFDFormat(CAN))
    {
        u32Prescaler = CAN_GetNominalBitRatePrescaler(CAN);
        /* 5Mbps for 100MHz clock */
        CAN_SetDataBitRatePrescaler(CAN, u32Prescaler);
```

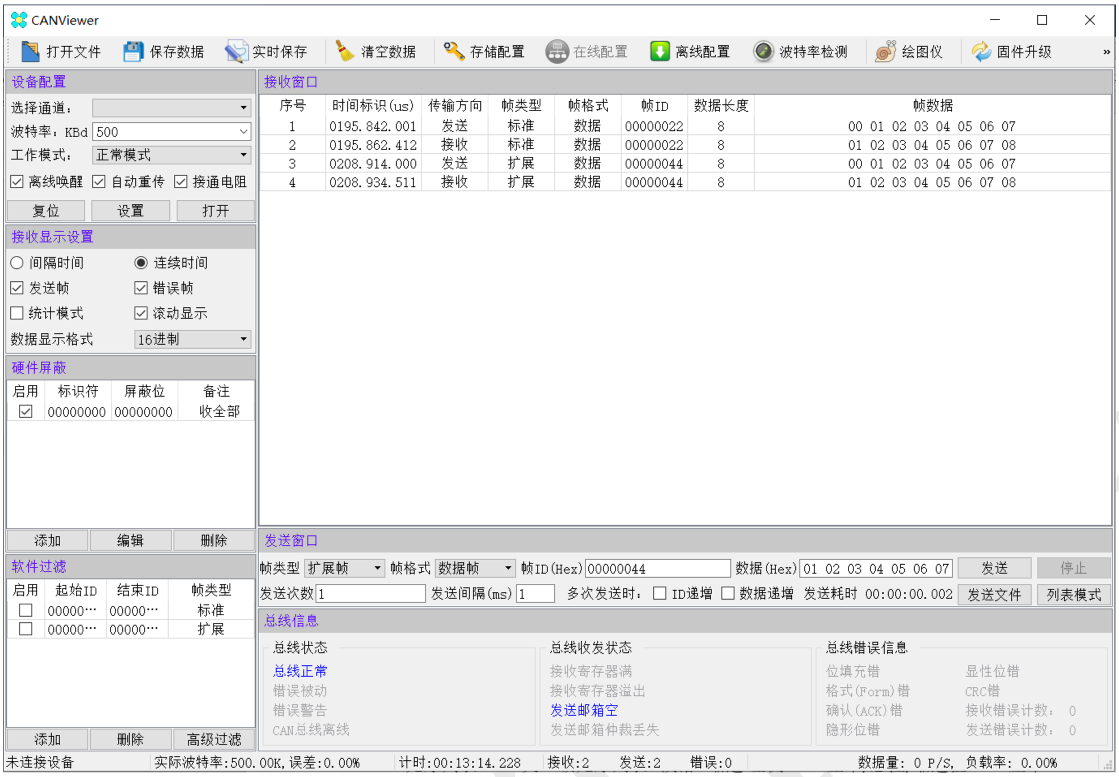
```
        CAN_SetDataBitPhaseBufferSegment1(CAN, 17 );
        CAN_SetDataBitPhaseBufferSegment2(CAN, 2 ); /* sample point: 80% */
        CAN_SetDataBitResyncJumpWidth    (CAN, 2 );
    }

    /* Calculate nominal phase baud rate */
    u32Data = CAN_GetNominalBitPhaseBufferSegment1(CAN) + 1;
    u32Data += CAN_GetNominalBitPhaseBufferSegment2(CAN);
    u32Data *= CAN_GetNominalBitRatePrescaler(CAN);
    f64NBps = CLOCK_GetModuleClock(CAN_MODULE) / u32Data;

    printf("[CAN]NBps: %f\n", f64NBps ) ;

    /* Calculate data phase baud rate */
    if ( CAN_IsEnableFDFormat(CAN))
    {
        u32Data = CAN_GetDataBitPhaseBufferSegment1(CAN) + 1;
        u32Data += CAN_GetDataBitPhaseBufferSegment2(CAN);
        u32Data *= CAN_GetDataBitRatePrescaler(CAN);
        f64DBps = CLOCK_GetModuleClock(CAN_MODULE) / u32Data;
        printf("[CAN]DBps: %f\n", f64DBps ) ;
    }

    /* Set delay befor bus-off recovery, based on nominal bit time */
    /* Set delay 1ms */
    u32Data = (uint32_t)(1.0e6 / ( 1.0e9 / f64NBps));
    if (u32Data > 65535)
    {
        u32Data = 65535;
    }

    CAN_SetBusOffRecoveryDelay(CAN, u32Data) ;

    /* Set transmitter delay compensation for FD frame */
    CAN_EnableTransmitterDelayCompensation(CAN);

    /* Set transmitter delay compensation offset and window for FD frame */
    u32DateSegment1 = CAN_GetDataBitPhaseBufferSegment1(CAN);
    CAN_SetTransmitterDelayOffset(CAN, u32DateSegment1 + 1 );
    CAN_SetTransmitterDelayWindow(CAN, u32DateSegment1 + 1 );

    /* Set message RAM parity check */
    CAN_EnableParityCheck(CAN);

    /* Set timestamp */
    CAN_EnableTimestamp(CAN);
}


static void CAN_Printf_Message(CAN_MessageTypeDef *msg)
{
    int i;

    if (msg->eType == CAN_FRAME_DATA)
    {
        printf("    ") ;
        for (i = 0 ; i < msg->u8DataLen; i++)
        {
            printf("0x%02X,", msg->pu8Data[i]);
            if (i % 8 == 7)
            {
                printf("\n    ");
            }
```

```
        }
    }

    printf("\n");
}


static void CAN_ProcessMessageData(CAN_MessageTypeDef *msg)
{
    int i;
    ErrorStatus status = SUCCESS;

    if (msg->eType == CAN_FRAME_DATA)
    {
        for (i = 0 ; i < msg->u8DataLen; i++)
        {
           au8TxBuffer[i] = msg->pu8Data[i] + 1;
        }
    }
}


static void CAN_SendReplyMessage(CAN_MessageTypeDef *msg)
{
    int i;

    message3.eDataDir    = CAN_MSG_DATA_TX;
    message3.eRmtRspEn   = DISABLE;
    message3.eIntEn      = ENABLE;
    message3.eEobEn      = DISABLE;
    message3.eBrs        = DISABLE;
    message3.eEsiCtlEn   = ENABLE;
    message3.eEsi        = DISABLE;

    message3.u32Id       = msg->u32Id;
    message3.u8MBoxId    = 2;
    message3.eFormat     = msg->eFormat;
    message3.eType       = CAN_FRAME_DATA;

    /* DATA FIELD */
    message3.u8DataLen   = msg->u8DataLen;
    message3.pu8Data     = au8TxBuffer;

    CAN_SetMessage(CAN, &message3);
    CAN_EnableMailbox(CAN, message3.u8MBoxId);

    /* Wait Reply Data frame message3 txed */
    while (CAN_GetMailboxControlInfo(CAN, message3.u8MBoxId,
CAN_MBOX_SET_MSG_NEW | CAN_MBOX_REQUEST_TX)) {}
}

/*******************************************************************************
 *
 * @brief     In this case, the CAN Receive Extended frame data.
 *
 ******************************************************************************/
int main(void)
{
    ErrorStatus status;

    CLOCK_InitWithRCO(CLOCK_CPU_100MHZ);

    Delay_Init();
```

```
    /*
     * Init the UART
     */
    PIN_SetChannel(PIN_GPIO10, PIN_GPIO10_UART0_TXD);
    PIN_SetChannel(PIN_GPIO11, PIN_GPIO11_UART0_RXD);
    UART_Init(UART0, 38400);

    printf("CAN: Receive Message\n");

    /* CAN Clk Enable */
    CAN_Clk_Enable();

    /* CAN Disable */
    CAN_Module_Disable();

    /* CAN Gpio Init */
    CAN_Gpio_Init();

    /* CAN config */
    CAN_Config();

    /* Init message RAM contents before enable CAN module */
    CAN_InitMessageRAM(CAN);

    /* CAN run */
    CAN_Enable(CAN);

    /* Init mailbox for receive message1 */
    message1.eDataDir    = CAN_MSG_DATA_RX;
    message1.eRmtRspEn   = DISABLE;
    message1.eIntEn      = ENABLE;
    message1.eEobEn      = DISABLE;
    message1.eOverwriteEn= DISABLE;//ENABLE;
    message1.eMaskRtrEn  = ENABLE;
    message1.eMaskIdeEn  = ENABLE;

    message1.u32Id       = 0x22;
    message1.u32IdMask   = 0xff;
    message1.u8MBoxId    = 0;
    message1.eFormat     = CAN_FORMAT_STD;
    message1.eType       = CAN_FRAME_DATA;

    /* DATA FIELD */
    message1.u8DataLen   = CAN_DATA_LEN;

    status = CAN_SetMessage(CAN, &message1);
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message1.u8MBoxId);
        return -1;
    }

    message1.pu8Data = (uint8_t *)(long)(& CAN-
>CANMBOX[message1.u8MBoxId].CANMBOXFDW[0]);

    CAN_EnableMailbox(CAN, message1.u8MBoxId);

    /* Wait receive CAN Standard Data frame message2 */
    while (!CAN_GetMailboxControlInfo(CAN, message1.u8MBoxId,
CAN_MBOX_SET_MSG_NEW)) {}

    CAN_GetMessage( CAN, &message1);
```

```
    #if DEBUG_INFO
    printf("CAN Standard Frame Data:\n") ;
    CAN_Printf_Message(&message1);
    #endif

    CAN_ProcessMessageData(&message1);
    CAN_SendReplyMessage(&message1);

    /* Init mailbox for receive message2 */
    message2.eDataDir    = CAN_MSG_DATA_RX;
    message2.eRmtRspEn   = DISABLE;
    message2.eIntEn      = ENABLE;
    message2.eEobEn      = DISABLE;
    message2.eOverwriteEn= ENABLE;
    message2.eMaskRtrEn  = ENABLE;
    message2.eMaskIdeEn  = ENABLE;

    message2.u32Id       = 0x44;
    message2.u32IdMask   = 0xff;
    message2.u8MBoxId    = 1;
    message2.eFormat     = CAN_FORMAT_EXT;
    message2.eType       = CAN_FRAME_DATA;

    /* DATA FIELD */
    message2.u8DataLen   = CAN_DATA_LEN;

    status = CAN_SetMessage(CAN, &message2);
    if (status == ERROR)
    {
       printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message2.u8MBoxId);
       return -1;
    }

    message2.pu8Data = (uint8_t *)(long)(& CAN-
>CANMBOX[message2.u8MBoxId].CANMBOXFDW[0]);

    CAN_EnableMailbox(CAN, message2.u8MBoxId);

    /* Wait receive CAN Extended Data frame message2 */
    while (!CAN_GetMailboxControlInfo(CAN, message2.u8MBoxId,
CAN_MBOX_SET_MSG_NEW)) {}

    CAN_GetMessage( CAN, &message2);

    #if DEBUG_INFO
    printf("CAN Extended Frame Data:\n") ;
    CAN_Printf_Message(&message2);
    #endif

    CAN_ProcessMessageData(&message2);
    CAN_SendReplyMessage(&message2);


    while(1)
    {

    }
}
```

图 3-1：采用经典 CAN 协议的通信结果



3.2 CANFD 协议

下面将展示采用 CANFD 协议的通信实例，总线上包含两个节点：上位机（连接 PC 的 USB CAN）和 SPD1179。上位机依次发送一条标准数据帧和一条扩展数据帧，SPD1179 将收到的每个 Byte 数据“+1”后，以同样的格式发回上位机。实例代码如下所示，通信结果如图 3-2 所示。

Example Code

```
#include <stdio.h>
#include <stdlib.h>

#if defined(SPD1179)
    #include "spd1179.h"
#else
    #include "spc1169.h"
#endif


#define DEBUG_INFO      1

#define CAN_DATA_LEN       8

uint8_t au8TxBuffer[64];

/* CAN Standard Data frame message1 */
CAN_MessageTypeDef message1;
/* CAN Extended Data frame message2 */
CAN_MessageTypeDef message2;
/* CAN reply data frame message3 */
CAN_MessageTypeDef message3;
```

```
static void CAN_Clk_Enable(void)
{
    /* CAN Clk Enable */
    CLOCK_SetModuleDiv(CAN_MODULE, 1);
    CLOCK_EnableModule(CAN_MODULE);
}


static void CAN_Module_Disable(void)
{
    /* Disable run */
    CAN_Disable(CAN) ;

    /* FIXME add timeout control */
    while (CAN_GetOperationMode(CAN) != CAN_OPERATE_STOP) {}
}

static void CAN_Gpio_Init(void)
{
    #if defined(SPD1179)
    PIN_SetChannel(PIN_GPIO8, PIN_GPIO8_CAN_TXD);
    PIN_SetChannel(PIN_GPIO9, PIN_GPIO9_CAN_RXD);
    PIN_SetPinAsGPIO(PIN_GPIO6);
    GPIO_SetPinDir(PIN_GPIO6, GPIO_OUTPUT);
    GPIO_SetLow(PIN_GPIO6); // Set CAN PHY to normal mode (STB Pin level low)
    PIN_SetPinAsGPIO(PIN_GPIO7);
    GPIO_SetPinDir(PIN_GPIO7, GPIO_OUTPUT);
    GPIO_SetHigh(PIN_GPIO7);    // Enable LDO
    #else
    PIN_SetChannel(PIN_GPIO38, PIN_GPIO38_CAN_TXD);
    PIN_SetChannel(PIN_GPIO39, PIN_GPIO39_CAN_RXD);
    #endif
}

static void CAN_Config(void)
{
    uint32_t u32Data ;
    uint32_t u32DateSegment1 ;

    double f64NBps ;
    double f64DBps ;

    uint32_t u32Prescaler;

    /* Set IO swap */
    CAN_DisablePinSwap(CAN) ;

    /* Set FDCAN fromat support */
    CAN_EnableFDFormat(CAN);

    /* Set ISO-CAN frame support */
    CAN_DisableNonIsoMode(CAN);

    /* Set protocol exception mode */
    CAN_EnableProtocolException(CAN);

    /* Set restricted mode */
    CAN_DisableRestrictedMode(CAN);

    /* Set monitor mode */
    CAN_DisableMonitorMode(CAN);

    /* Set transmission pause function */
```

```
    CAN_EnableTransmissionPause(CAN);

    /* Set RXD edge filter during integration status */
    CAN_EnableEdgeFilter(CAN);

    /* Enable re-tran message afte lost arbitration or error */
    CAN_EnableAutoRetransmission(CAN);

    /* Enable auto start bus-off recovery sequence after bus-off */
    CAN_EnableAutoBusOn(CAN);

    /* Set nominal bit baud rate
     * 1MHz for 100MHz clock
     **/
    CAN_SetNominalBitRatePrescaler(CAN, 5);
    CAN_SetNominalBitPhaseBufferSegment1(CAN, 14);
    CAN_SetNominalBitPhaseBufferSegment2(CAN, 5); /* sample point: 83% */
    CAN_SetNominalBitResyncJumpWidth(CAN, 1);

    /* Set data bit baud rate */
    if ( CAN_IsEnableFDFormat(CAN))
    {
        u32Prescaler = CAN_GetNominalBitRatePrescaler(CAN);
        /* 5Mbps for 100MHz clock */
        CAN_SetDataBitRatePrescaler(CAN, u32Prescaler);
        CAN_SetDataBitPhaseBufferSegment1(CAN, 2 );
        CAN_SetDataBitPhaseBufferSegment2(CAN, 1 ); /* sample point: 80% */
        CAN_SetDataBitResyncJumpWidth    (CAN, 1 );
    }

    /* Calculate nominal phase baud rate */
    u32Data = CAN_GetNominalBitPhaseBufferSegment1(CAN) + 1;
    u32Data += CAN_GetNominalBitPhaseBufferSegment2(CAN);
    u32Data *= CAN_GetNominalBitRatePrescaler(CAN);
    f64NBps = CLOCK_GetModuleClock(CAN_MODULE) / u32Data;

    printf("[CAN]NBps: %f\n", f64NBps ) ;

    /* Calculate data phase baud rate */
    if ( CAN_IsEnableFDFormat(CAN))
    {
        u32Data = CAN_GetDataBitPhaseBufferSegment1(CAN) + 1;
        u32Data += CAN_GetDataBitPhaseBufferSegment2(CAN);
        u32Data *= CAN_GetDataBitRatePrescaler(CAN);
        f64DBps = CLOCK_GetModuleClock(CAN_MODULE) / u32Data;
        printf("[CAN]DBps: %f\n", f64DBps ) ;
    }

    /* Set delay befor bus-off recovery, based on nominal bit time */
    /* Set delay 1ms */
    u32Data = (uint32_t)(1.0e6 / ( 1.0e9 / f64NBps));
    if (u32Data > 65535)
    {
        u32Data = 65535;
    }

    CAN_SetBusOffRecoveryDelay(CAN, u32Data) ;

    /* Set transmitter delay compensation for FD frame */
    CAN_EnableTransmitterDelayCompensation(CAN);

    /* Set transmitter delay compensation offset and window for FD frame */
    u32DateSegment1 = CAN_GetDataBitPhaseBufferSegment1(CAN);
```

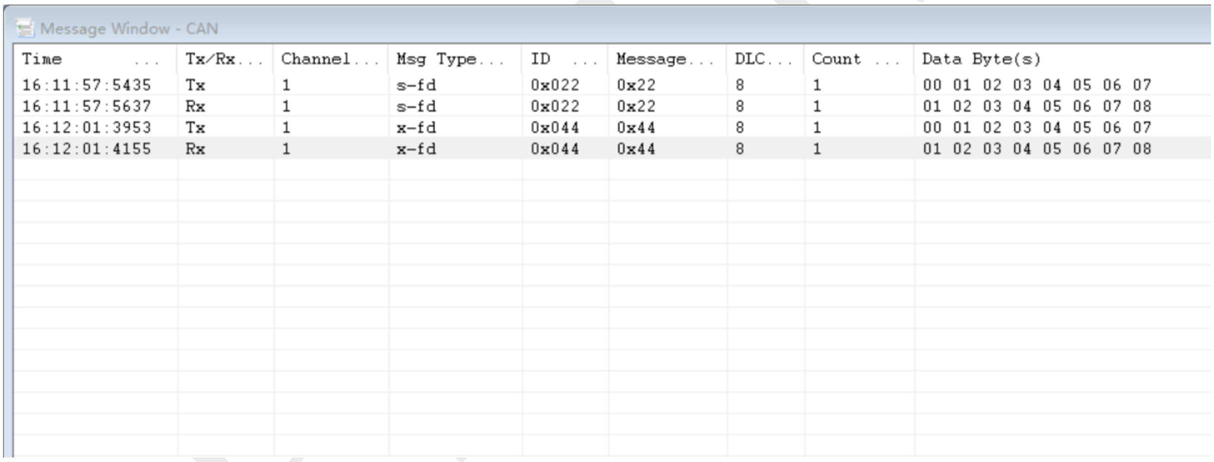
```
    CAN_SetTransmitterDelayOffset(CAN, u32DateSegment1 + 1 );
    CAN_SetTransmitterDelayWindow(CAN, u32DateSegment1 + 1 );

    /* Set message RAM parity check */
    CAN_EnableParityCheck(CAN);

    /* Set timestamp */
    CAN_EnableTimestamp(CAN);
}


static void CAN_Printf_Message(CAN_MessageTypeDef *msg)
{
    int i;

    if (msg->eType == CAN_FRAME_DATA)
    {
        printf("    ") ;
        for (i = 0 ; i < msg->u8DataLen; i++)
        {
            printf("0x%02X,", msg->pu8Data[i]);
            if (i % 8 == 7)
            {
                printf("\n    ");
            }
        }
    }

    printf("\n");
}


static void CAN_ProcessMessageData(CAN_MessageTypeDef *msg)
{
    int i;
    ErrorStatus status = SUCCESS;

    if (msg->eType == CAN_FRAME_DATA)
    {
        for (i = 0 ; i < msg->u8DataLen; i++)
        {
           au8TxBuffer[i] = msg->pu8Data[i] + 1;
        }
    }
}


static void CAN_SendReplyMessage(CAN_MessageTypeDef *msg)
{
    int i;

    message3.eDataDir    = CAN_MSG_DATA_TX;
    message3.eRmtRspEn    = DISABLE;
    message3.eIntEn      = ENABLE;
    message3.eEobEn      = DISABLE;
    message3.eBrs        = DISABLE;
    message3.eEsiCtlEn    = ENABLE;
    message3.eEsi        = DISABLE;

    message3.u32Id       = msg->u32Id;
    message3.u8MBoxId     = 2;
    message3.eFormat     = msg->eFormat;
    message3.eType       = CAN_FRAME_DATA;
```

```
    /* DATA FIELD */
    message3.u8DataLen   = msg->u8DataLen;
    message3.pu8Data     = au8TxBuffer;

    CAN_SetMessage(CAN, &message3);
    CAN_EnableMailbox(CAN, message3.u8MBoxId);

    /* Wait Reply Data frame message3 txed */
    while (CAN_GetMailboxControlInfo(CAN, message3.u8MBoxId,
CAN_MBOX_SET_MSG_NEW | CAN_MBOX_REQUEST_TX)) {}
}

/*******************************************************************************
 *
 * @brief     In this case, the CAN Receive Extended frame data.
 *
 ******************************************************************************/
int main(void)
{
    ErrorStatus status;

    CLOCK_InitWithRCO(CLOCK_CPU_100MHZ);

    Delay_Init();

    /*
     * Init the UART
     */
    PIN_SetChannel(PIN_GPIO10, PIN_GPIO10_UART0_TXD);
    PIN_SetChannel(PIN_GPIO11, PIN_GPIO11_UART0_RXD);
    UART_Init(UART0, 38400);

    printf("CAN: Receive Message\n");

    /* CAN Clk Enable */
    CAN_Clk_Enable();

    /* CAN Disable */
    CAN_Module_Disable();

    /* CAN Gpio Init */
    CAN_Gpio_Init();

    /* CAN config */
    CAN_Config();

    /* Init message RAM contents before enable CAN module */
    CAN_InitMessageRAM(CAN);

    /* CAN run */
    CAN_Enable(CAN);

    /* Init mailbox for receive message1 */
    message1.eDataDir     = CAN_MSG_DATA_RX;
    message1.eRmtRspEn    = DISABLE;
    message1.eIntEn       = ENABLE;
    message1.eEobEn       = DISABLE;
    message1.eOverwriteEn= DISABLE;//ENABLE;
    message1.eMaskRtrEn   = ENABLE;
    message1.eMaskIdeEn   = ENABLE;

    message1.u32Id        = 0x22;
```

```
    message1.u32IdMask   = 0xff;
    message1.u8MBoxId    = 0;
    message1.eFormat     = CAN_FORMAT_FD_STD;
    message1.eType       = CAN_FRAME_DATA;

    /* DATA FIELD */
    message1.u8DataLen   = CAN_DATA_LEN;

    status = CAN_SetMessage(CAN, &message1);
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message1.u8MBoxId);
        return -1;
    }

    message1.pu8Data = (uint8_t *)(long)(& CAN-
>CANMBOX[message1.u8MBoxId].CANMBOXFDW[0]);

    CAN_EnableMailbox(CAN, message1.u8MBoxId);

    /* Wait receive CAN Standard Data frame message2 */
    while (!CAN_GetMailboxControlInfo(CAN, message1.u8MBoxId,
CAN_MBOX_SET_MSG_NEW)) {}

    CAN_GetMessage( CAN, &message1);

    #if DEBUG_INFO
    printf("CAN Standard Frame Data:\n") ;
    CAN_Printf_Message(&message1);
    #endif

    CAN_ProcessMessageData(&message1);
    CAN_SendReplyMessage(&message1);

    /* Init mailbox for receive message2 */
    message2.eDataDir    = CAN_MSG_DATA_RX;
    message2.eRmtRspEn   = DISABLE;
    message2.eIntEn      = ENABLE;
    message2.eEobEn      = DISABLE;
    message2.eOverwriteEn= ENABLE;
    message2.eMaskRtrEn  = ENABLE;
    message2.eMaskIdeEn  = ENABLE;

    message2.u32Id       = 0x44;
    message2.u32IdMask   = 0xff;
    message2.u8MBoxId    = 1;
    message2.eFormat     = CAN_FORMAT_FD_EXT;
    message2.eType       = CAN_FRAME_DATA;

    /* DATA FIELD */
    message2.u8DataLen   = CAN_DATA_LEN;

    status = CAN_SetMessage(CAN, &message2);
    if (status == ERROR)
    {
        printf("[CAN][u8MBoxId:%2d] Set MBOX failed", message2.u8MBoxId);
        return -1;
    }

    message2.pu8Data = (uint8_t *)(long)(& CAN-
>CANMBOX[message2.u8MBoxId].CANMBOXFDW[0]);

    CAN_EnableMailbox(CAN, message2.u8MBoxId);
```

```
    /* Wait receive CAN Extended Data frame message2 */
    while (!CAN_GetMailboxControlInfo(CAN, message2.u8MBoxId,
CAN_MBOX_SET_MSG_NEW)) {}

    CAN_GetMessage( CAN, &message2);

    #if DEBUG_INFO
    printf("CAN Extended Frame Data:\n") ;
    CAN_Printf_Message(&message2);
    #endif

    CAN_ProcessMessageData(&message2);
    CAN_SendReplyMessage(&message2);


    while(1)
    {

    }
}
```

图 3-2：采用 CANFD 的通信结果



Message Window - CAN

Time	Tx/Rx	Channel	Msg Type	ID	Message	DLC	Count	Data Byte(s)
16:11:57:5435	Tx	1	s-fd	0x022	0x22	8	1	00 01 02 03 04 05 06 07
16:11:57:5637	Rx	1	s-fd	0x022	0x22	8	1	01 02 03 04 05 06 07 08
16:12:01:3953	Tx	1	x-fd	0x044	0x44	8	1	00 01 02 03 04 05 06 07
16:12:01:4155	Rx	1	x-fd	0x044	0x44	8	1	01 02 03 04 05 06 07 08